



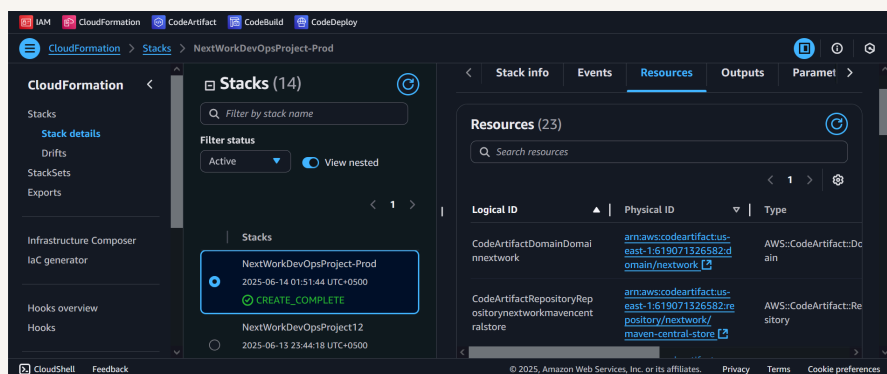
nextwork.org

Infrastructure as Code with CloudFormation



Muhammad Sadique

<https://www.linkedin.com/in/muhammad-sadique->





Introducing Today's Project!

In this project, I will demonstrate how to use CloudFormation to turn my web app infrastructure into one handy template! I'm doing this project to learn AWS CloudFormation, i.e. an Infrastructure as Code (IaC) tool used in DevOps practices.

Key tools and concepts

Service I used was CloudFormation. The key concept that I learnt was how IaC works in AWS and how IaC Generator creates a template of running AWS resources

Project reflection

This project took me approximately a day. The most challenging part was to troubleshoot the template file in yaml, eliminating circular dependencies and hard coded values referred inside CloudFormation Template.

This project is part six of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project tomorrow inshALLAH.



Generating a CloudFormation Template

The IaC Generator is a tool in CloudFormation that helps make writing templates much, much faster. It scans our AWS account, discovers all our resources, and generates the code for the resources we want to put in a CloudFormation template. This means we won't have to write code from scratch! It works in a three step process where: 1. We scan all the resources in our AWS account 2. We create a template, which is a bundle of scanned resources that w'd like to deploy and manage together. 3. We import the template into CloudFormation to deploy the resources in one go.

A CloudFormation template lets us create and configure AWS resources (e.g. EC2 instances, CodeBuild projects, CodeArtifact repositories, etc.) as code instead of clicking around in the AWS console. The resource that i could add to template include:
AWS::EC2::Instance AWS::EC2::NetworkInterface AWS::EC2::SecurityGroup
AWS::EC2::Subnet AWS::EC2::Volume AWS::EC2::VolumeAttachment AWS::EC2::VPC
3x AWS::IAM::ManagedPolicy

The resources I could not add to my template were CodeBuild Project & CodeDeploy Deployment Group because the CloudFormation template struggles with resources requiring complex configurations.



The screenshot shows the AWS CloudFormation console interface. The left sidebar contains navigation options: Stacks, StackSets, Exports, Infrastructure Composer, **IaC generator**, Hooks overview, and Hooks. The main area displays the 'nextwork-devops-cicd-iaC-Template' template. The template content is as follows:

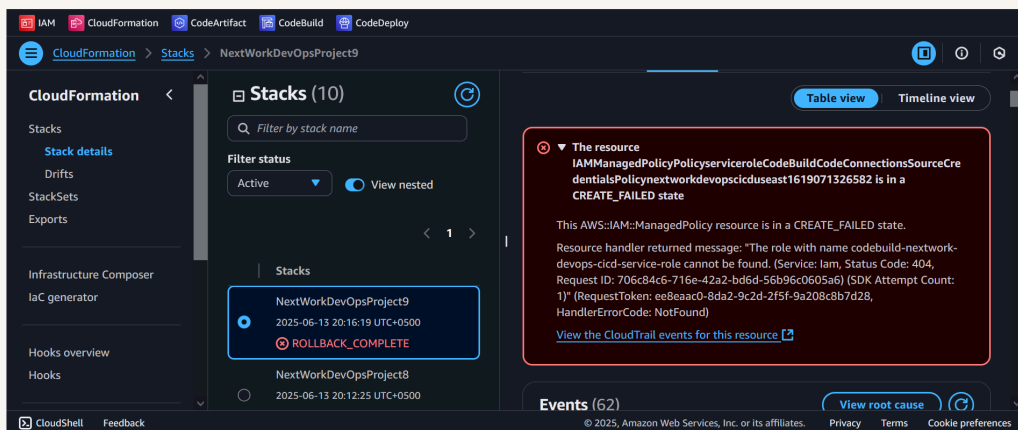
```
1 Metadata:
2   AWS::IAM::Role:
3     IAC_Generator: arn:aws:cloudformation:us-east-1:619071326582:generatedTemplate/10543518-6b1
4 Resources:
5   IAMRoleEC2InstanceNextWorkCicd:
6     UpdateReplacePolicy: Delete
7     Type: AWS::IAM::Role
8     DeletionPolicy: Delete
9     Properties:
10      Path: /
11      ManagedPolicyArns:
12        - !Ref IAMManagedPolicyPolicycodeartifactnextworkconsumerpolicy
13      MaxSessionDuration: 3600
14      RoleName: EC2-instance-nextwork-cicd
15      Description: Allows EC2 instances to access services related to the NextWork
16      CI/CD pipeline series.
```




Template Testing

Before testing my template i deleted my CI/CD resources so that while template try to create a resource, there should not be any resource already in AWS account with the similar name. So if i do not delete my resources, my template will fail to run.

I tested my template by creating a new stack. The results of my first test was "ROLL_BACK_COMPLETE" because of following error message: This AWS::IAM::ManagedPolicy resource is in a CREATE_FAILED state. Resource handler returned message: "The role with name codebuild-nextwork-devops-cicd-service-role cannot be found. (Service: iam, Status Code: 404, Request ID: 706c84c6-716e-42a2-bd6d-56b96c0605a6) (SDK Attempt Count: 1)" (RequestToken: ee8eaac0-8da2-9c2d-2f5f-9a208c8b7d28, HandlerErrorCode: NotFound)





DependsOn

To resolve the error, I added the following line under

```
"IAMManagedPolicyPolicy servicerolesCodeBuildCodeConnectionsSourceCredentialsPolicynextworkdevopscicduseast1619071326582:" DependsOn: "codebuild-nextwork-devops-cicd-service-role"
```

It would make the policy dependent on creation of role. The policy creation will wait until the required role is created.

The DependsOn line was added to four different parts of my template:

```
IAMManagedPolicyPolicy servicerolesCodeBuild
```

```
IAMManagedPolicyPolicy servicerolesCodeBuildCloudWatchLogsPolicynextworkdevopscicduseast1
```

```
IAMManagedPolicyPolicy servicerolesCodeBuildBasePolicynextworkdevopscicduseast1
```

For CodeArtifact policy under following section

```
IAMManagedPolicyPolicy codeartifactnextworkconsumerpolicy
```

I added following lines:

```
DependsOn: - "codebuild-nextwork-devops-cicd-service-role" -
```

```
"IAMRoleEC2instancenextworkcicd"
```



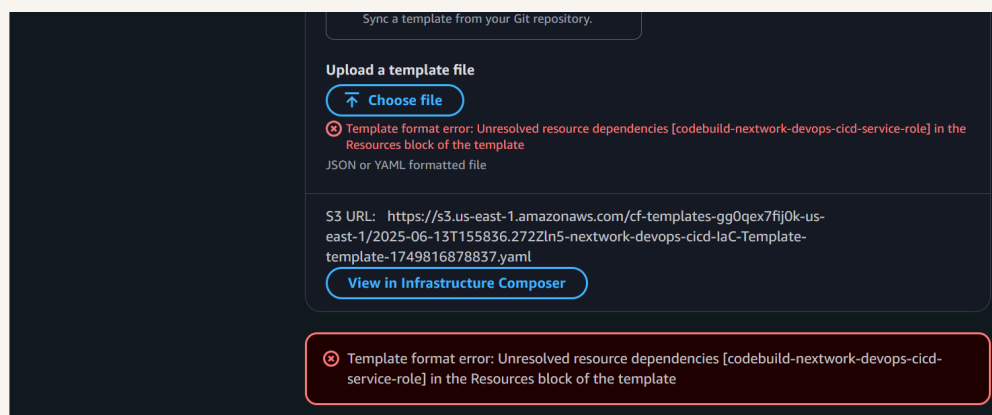
```
IAMManagedPolicyPolicyServiceRoleCodeBuildCodeConnectionsSourceCredentialsPolicynextworkdevopscicduseast16190713265
UpdateReplacePolicy: "Delete"
Type: "AWS::IAM::ManagedPolicy"
DeletionPolicy: "Delete"
DependsOn: "codebuild-nextwork-devops-cicd-service-role"
Properties:
```



Circular Dependencies

I gave my CloudFormation template another test! But this time it said that my template had a circular dependencies. Meaning that policies are roles are dependent on each other for creation, so none of those creates and the loop goes on.

To fix this error, I deleted ManagedPolicyArns as they were creating circular dependency.





Manual Additions

In a project extension, I manually defined two more resources: CodeBuild Project & CodeDeploy Deployment Group

I also had to make sure the references were consistent in this template, so I edited value inside placeholder in following blocks: DependsOn: - "
<YOUR_CODEBUILD_SERVICE_ROLE_ID>" - "<YOUR_S3_BUCKET_ID>" Source: Type: GITHUB Location: !Sub "https://github.com/\${GitHubRepoOwner}/\${GitHubRepo}" BuildSpec: buildspec.yml Artifacts: Type: S3 Name: nextwork-web-build.zip Packaging: ZIP Location: !Ref <YOUR_S3_BUCKET_ID> ServiceRole: !GetAtt <YOUR_CODEBUILD_SERVICE_ROLE_ID>.Arn

I also introduced Parameters, which are GitHubRepoOwner and GitHubRepo. Parameters make your CloudFormation template reusable and flexible. Instead of hardcoding values like GitHub usernames, you can parameterize them. This lets you use the same template for different environments or repositories without modifying the template itself.

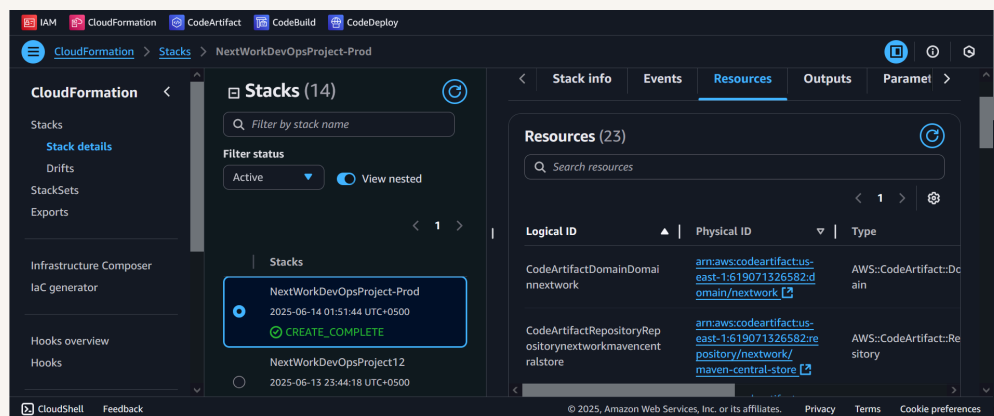


```
network-DevOps CloudFormation-2.yaml | Working-network-DevOps-CloudFormation-2.yaml
network-DevOps CloudFormation-2.yaml
Resources:
  CodeBuildProject:
    455: CodeBuildProject
    456: CodeBuildProject
    457: Type: AWS::CodeBuild::Project
    458: Properties:
    459:   - IAMRole: iam-role-network-devops-cid-service-role # Logical ID of the Role
    460:     - S3Bucket: s3-bucket-network-devops-cid-service-role # Logical ID of the Bucket
    461:   Name: network-devops-cid
    462:   Description: Build project for NextWork web application
    463:   Source:
    464:     Type: GITHUB
    465:     Location: /sub "https://github.com/git-muhammad-sadique/100/network-web-project.git"
    466:     BuildSpec: buildspec.yml
    467:   Artifacts:
    468:     Type: S3
    469:     Name: network-web-build.zip
    470:     Packaging: ZIP
    471:     Location: /sub s3-bucket-network-devops-cid-service-role
    472:     Path: /build
    473:   Environment:
    474:     Type: LAMBDA_CONTAINER
    475:     ComputeType: BUILD_GENERAL1_SMALL
    476:     Image: aws/codebuild/amazonlinux2-x86_64-standard:corretto8
    477:     ServiceRole: !GetAtt iam-role-network-devops-cid-service-role.Arn
    478:     LogsConfig:
    479:       CloudWatchLogs:
    480:         GroupName: /aws/codebuild/network-devops-cid
    481:         Status: ENABLED
    482:         StreamName: webapp
    483:
```



Success!

I could verify all the deployed resources by visiting "Resource" tab in CloudFormation stack.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

